

Towards Transformer-Based Flow Volume Prediction in Network Traffic

Samara Mayhoub*, Mirko Schiavone*, Chuan Heng Foh*, Mohammad Shojafar*

*5GIC/6GIC, Institute for Communication Systems (ICS), University of Surrey, Guildford, UK

Email: {s.mayhoub, ms04061, c.foh, m.shojafar}@surrey.ac.uk

Abstract—Accurately predicting network flow volume using early packet-level features is critical for real-time applications such as resource allocation, bottleneck prediction, anomaly detection and more. In this paper, we investigate the suitability of Transformer-based architectures for the flow volume regression task. Specifically, we fine-tune two foundation models, NetFound and YaTC, originally pre-trained to learn traffic representations for classification tasks, and adapt them for flow volume regression. Additionally, we train a lightweight Transformer model, and compare the predictive performance and complexity of all Transformer-based models to a simple Multi-Layer Perceptron (MLP) model. Fine-tuning and training are conducted on two public datasets: MAWI and CIC-IDS-2017. Results demonstrate that Transformer-based models significantly outperform the MLP model. The lightweight Transformer achieves the best R^2 score of 0.963 on CIC-IDS-2017, while NetFound achieves the best performance on the real-world traffic dataset MAWI with an R^2 of 0.812. These findings highlight the effectiveness of Transformers in capturing complex relationships in early packet-level features of network traffic for flow volume regression task.

Index Terms—Transformer, Foundation Models, Network Traffic Analysis, Flow Volume Prediction, Deep Learning

I. INTRODUCTION

Predicting traffic volume is critical for the efficient management of modern communication systems, particularly in dynamic and latency-sensitive environments such as the Internet of Things (IoT) and Vehicular Networks (V2X). These networks generate diverse and high-volume traffic, requiring intelligent mechanisms for early prediction to support tasks such as bandwidth provisioning, congestion control, and anomaly detection. In V2X systems, which enable real-time data exchange between vehicles and infrastructure timely and precise flow volume prediction is essential to maintain Quality-of-Service (QoS) and support autonomous operations.

Neural networks have been widely explored for traffic characteristics prediction due to their ability to model complex, non-linear relationships in traffic data. However, traditional neural networks often struggle with long-term dependencies and scalability. These limitations have led to growing interest in transformer-based models, which offer superior performance by leveraging attention mechanisms to capture both spatial and temporal correlations more effectively. Large Language Models (LLMs) and Transformer-based architectures have improved Natural Language Processing (NLP) applications through their ability to capture complex dependencies using the revolutionary idea of self-attention [1]. While originally Transformer-based architectures were developed for

language tasks such as translation and text classification, recent advances have shown their applications in domains beyond NLP. In particular, in telecommunications, recent work demonstrates how LLMs and Transformers can be adapted for various traffic classification and network optimization applications [2], [3]. For example, SecurityBERT [4] adapts Bidirectional Encoder Representations from Transformers (BERT) model [5] for encrypted IoT traffic classification. Similarly, NetFound [6] introduces a Transformer-based foundation model that learns traffic representations. Also, YaTC [7] uses vision Transformer technique on packets image representations. On the other hand, LLMs have also been used to explain decisions made by an ML-based anomaly detector in Open Radio Access Network (O-RAN) [8]. Moreover, the Time-series Transformer (TimeGPT) [9] was introduced to solve regression problems in telecommunications including, channel state prediction, user mobility prediction and Network traffic prediction.

Together, these studies show that LLMs and Transformers are no longer limited to language processing applications. Instead, they are being widely adapted to solve network-specific tasks, including traffic classification, anomaly detection, throughput prediction and more others, making them promising techniques for real-time network intelligence in future 6G networks.

Motivated by recent successes of leveraging Transformer-based models in classification tasks in networks, this work explores their suitability for regression tasks such as flow volume regression. While foundation models like NetFound and YaTC have been pre-trained to learn traffic representations, their adaptation to regression tasks remains underexplored. We investigate this by fine-tuning both models for predicting flow volume. In addition, we introduce and train a lightweight Transformer model and evaluate all the models against a simple Multi-Layer Perceptron (MLP) model. The MLP is used as a strong low-complexity baseline commonly adopted in network traffic analysis studies [10].

This paper contributes to the emerging research on Transformers for network traffic applications as follows:

- Investigate the effectiveness of two pre-trained Transformer-based models, NetFound and YaTC, originally designed to learn meaningful representations of network traffic for classification tasks. We fine-tune these models for flow volume regression task by replacing their classification head with a MLP-based regressor head.

- Design and train a lightweight Transformer model for flow volume prediction from early packet-level features, aiming to balance prediction accuracy with efficiency for resource-constrained networks, such as IoT.
- Evaluate all the Transformer-based models and the simple MLP regressor using two public datasets: CIC-IDS-2017 [11], [12] and MAWI [13], [14]. The results show the predictive advantages of Transformer architectures while demonstrating that a lightweight Transformer can achieve competitive accuracy with reduced complexity.

To our knowledge, this paper presents the first empirical evidence that traffic-specific foundation models—pre-trained for classification—can be repurposed without full re-training for accurate flow-volume *regression*, extending their applicability beyond the prevailing security-centric use cases.

The remainder of this paper is structured as follows: Section II provides background and related work. Section III formulates the flow volume prediction problem. Section IV details the training procedures. Section V describes the experimental setup. Section VI presents the results and discussion. Section VII highlights known limitations and future work. Finally, Section VIII concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Transformer-Based Models

The fundamental component of LLM and many foundation models is the Transformer architecture [1]. It is based on the attention mechanism with two main components: an encoder, which extracts the features and understands the relationships within the input data; a decoder, which generates outputs while considering the encoded input representation and prior outputs. However, models like Generative Pre-trained Transformer (GPT) [15] use only the decoder and is optimized for text generation tasks, while BERT uses only the encoder and is suited for classification tasks.

Before processing, the text input (in LLM) goes through key pre-processing steps: Tokenization, which converts raw text into smaller meaningful units called tokens and then maps each token to a unique integer ID (a numerical representation) based on a predefined vocabulary; Embeddings, which maps each token ID to a high-dimensional vector; and Positional Encoding, which adds a special numerical pattern to each vector to retain the order of the tokens in the sequence.

In the Transformer model, input tokens are first linearly transformed into three matrices: Query (Q), Key (K), and Value (V). Then, the attention mechanism computes the relevance of each token to every other token using dot products of Q and K, which are then used to weigh the corresponding V vectors. This enables the model to capture long-range dependencies and contextual relationships [1]. In the context of network traffic, such capabilities are critical for representing packet flows where temporal and semantic dependencies span multiple levels (packet-level, byte-level, and field-level) in a flow [6], [7], [16], [17]. Unlike classical time-series forecasting (predicting the *next* observed point), our task infers an

unseen quantity: the eventual flow volume, using only the first few packets, whose protocol-specific patterns make network traffic fundamentally different from generic sequential data.

To answer the question of how Transformer-based foundation models are trained, the training process typically involves two main phases, as illustrated in Figure 1:

- *Pre-training*: This phase leverages a massive corpus of unlabeled data and employs *self-supervised learning* by considering pseudo-tasks like masked token prediction, to enable the model to learn general patterns and dependencies within the input data [18]. This approach enables the use of large datasets without requiring manual labeling — a key advantage of transfer learning, especially in telecommunications, where labeled datasets are often scarce. However, it is computationally expensive and time-consuming.
- *Fine-tuning*: This phase adapts the pre-trained model to a specific *downstream task* (e.g., classification, regression, translation) using a smaller, labeled dataset. Since the model has already learned general language or data patterns during pre-training, fine-tuning requires fewer computational resources and less data, making it more efficient and task-focused.

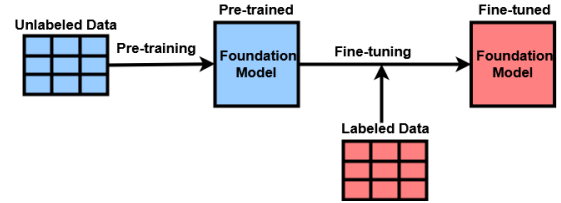


Fig. 1: Training phases of Transformer-based Foundation Model

The first option for working with Transformer-based foundation models or LLMs is to use a pre-trained and finetuned foundation model as-is for downstream tasks without additional training. This approach is known as zero-shot setting [19], [20] (by using, for example, OpenAI GPT, Gemini, etc.). The second option is to fine-tune an existing pre-trained LLM or a Transformer-based foundation model to specific task using a specific dataset and leveraging open access platforms, such as Hugging Face, OpenAI, Google Colab, and others. The third option is to build a LLM model or a Transformer-based model from scratch and pre-train it on your own dataset.

Some of the popular LLMs for fine-tuning applications: GPT (GPT-3, GPT-4), BERT and others. Additionally, platforms like Hugging Face [21] offer many open-access pre-trained Transformer-based models for various tasks, such as text, image processing, and most recently for network traffic related tasks.

B. Traffic-Specific Transformer-Based Models

Transformers have increasingly been applied to network traffic analysis due to their capability to capture complex dependencies across the raw packets. Existing methods vary

in how they encode input data (packet-level data), the self-supervised learning strategies they employ during pre-training phase, and the architecture of their Transformer.

Among the notable efforts are NetFound [6] and YaTC [7], which have demonstrated strong performance across classification tasks. *NetFound* is a network-specific foundation model that employs protocol-aware tokenization and a hierarchical Transformer architecture. It is pre-trained using a self-supervised objective: Masked Token Prediction, inspired by the original BERT pre-training objective, allowing it to learn generalized representations from raw packet flows. NetFound has been fine-tuned for several downstream tasks, including encrypted traffic classification and attack detection. *YaTC*, on the other hand, uses a Vision Transformer (ViT)-based architecture [22] for traffic classification. It represents network flows as grayscale Multi-level Flow Representation (MFR) matrices, which encode byte-level, packet-level, and flow-level information in a structured 2D format. YaTC is pre-trained using a self-supervised masked autoencoding strategy inspired by Masked Autoencoders, enabling it to reconstruct masked image patches and thereby learn meaningful latent representations of network traffic. It has been evaluated on multiple classification tasks, including encrypted traffic classification, and IoT intrusion detection. Additionally, *ET-BERT* [16] introduces a Transformer-based model for encrypted traffic classification by treating each burst of packets as a semantic unit (datagram) and learning contextualized representations. It employs two self-supervised pre-training objectives: Masked Burst Modeling and Same-Origin Burst Prediction, capturing both intra-burst byte-level dependencies and inter-burst transmission patterns. In contrast, *DeX* [23] is a lightweight Transformer-based model designed for real-time throughput prediction in video streaming scenarios. It leverages a Transformer encoder to jointly predict average and extreme throughput values.

The mentioned models show the growing interest in using transformers to classification tasks in network data. Here, we focus on adapting NetFound and YaTC to flow volume predicting task, exploring whether their learned representations can generalize beyond classification and how they compare to our lightweight Transformer model and to a MLP model. The MLP represents a well-established architecture with minimal inductive bias, and provides a practical reference point to quantify the benefits of more complex models. Its inclusion allows us to assess whether Transformer-based improvements justify the additional computational cost in this context. While a broad benchmark including Convolutional Neural Networks (CNNs) would be informative, we purposely confine the study to Transformer variants and a parameter-matched MLP in order to isolate the benefit of *pre-training and self-attention* on this new regression task, with NetFound serving as the current traffic-specific state-of-the-art reference point.

III. PROBLEM DEFINITION

In telecommunication, predicting the volume of data transferred in a network flow (in bytes) is essential for tasks such

as bottleneck prediction, early anomaly detection, and more others. Flow volume prediction involves estimating the total number of bytes transmitted within a network flow, using only partial or early packet-level information. This is valuable for real-time applications where complete flow information may not be available during early stages of transmission. Formally, let $\mathcal{F} = \{(X_i, V_i)\}_{i=1}^N$ be a dataset of N flows, where $X_i \in \mathbb{R}^{m_i \times d}$ is the matrix of the first m_i packet-level feature vectors of flow i ($m_i \leq M$ and d is the feature dimension) and $V_i \in \mathbb{R}_{>0}$ is the ground-truth total bytes exchanged over the *entire* flow. The goal is to learn a parameterised function $f_\theta : \mathbb{R}^{M \times d} \rightarrow \mathbb{R}_{>0}$ that outputs $\hat{V}_i = f_\theta(X_i)$, minimising the mean-squared-error loss $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (V_i - \hat{V}_i)^2$.

Traditional deep learning methods of traffic predictions rely on handcrafted features or statistical models [24], which often fail to capture the complex dependencies within packet flows. Recently, Transformers have demonstrated strong capabilities in learning rich representations from raw network traffic data. However, most existing models have been pre-trained for classification-oriented tasks such as application identification, encrypted traffic categorization, or intrusion detection.

In this work, we address network flow volume regression task, using early flow features. Our goal is to evaluate the effectiveness of Transformer-based models, particularly those pre-trained on classification tasks, for flow volume regression. Specifically, we aim to answer the following research questions:

- Can existing traffic-specific and pre-trained Transformer models (NetFound and YaTC), when fine-tuned appropriately, generalize to predict the flow volume (in bytes) using only the first few packets of the flow?
- How does a lightweight Transformer model trained on a specific task perform in comparison to pre-trained models and a simple MLP model?

We approach this by fine-tuning NetFound and YaTC and training a lightweight Transformer. Our experiments on two public datasets shows the suitability of these models for real-time flow volume prediction.

IV. METHODOLOGY

This section describes the used methods for predicting traffic flow volume. We fine-tune two pre-trained Transformer-based models, NetFound and YaTC. Additionally, we develop a lightweight Transformer and a MLP regressor. All models use a shared MLP-based regression head.

A. MLP Regression Head

To perform the regression task, we designed a simple MLP regression head that maps the extracted features from the pre-trained Transformer-based models (NetFound, YaTC, and lightweight Transformer) to a continuous output (i.e. flow volume in bytes). The architecture consists of three fully connected (Linear) layers with two LeakyReLU activation functions and dropout for regularization. This head replaces the original classification heads in NetFound, YaTC, and also serves as the final layer in our lightweight Transformer.

B. Fine-Tuning NetFound

We load the pre-trained model `netFound-640M-base` from hugging Face platform [25]. NetFound adopts a hierarchical Transformer architecture with 24 Transformer encoder layers, 16 attention heads, and 1024 hidden size, and was pre-trained on over 450 million real-world flows from the University of California network traffic.

To adapt NetFound for our regression task, we replace its original classification head with the Regressor head (detailed in Section IV-A). During fine-tuning, the base NetFound model is frozen. Moreover, We used the official NetFound pre-processing pipeline to process raw PCAP files into burst-level sequences. Each flow sample includes features such as tokenized burst content, packet direction, byte counts, IATs, packet counts, protocol type, and flow duration. These features are embedded and processed in batches by the frozen base model, followed by attention-based pooling. The resulting representations are passed to the regression head, which is trained by minimizing a function loss, as shown in Algorithm 1.

Algorithm 1 Generic Fine-Tuning Loop for Flow-Volume Regression

- 1: **Input:** Pre-processed flow batches $\mathcal{X}$, volume labels $\mathcal{V}$, pre-trained backbone $M \in \{\text{NetFound, YaTC}\}$, trainable regressor g_ϕ , loss $\mathcal{L}$
 - 2: Freeze all parameters of M (optionally unfreeze input-embedding layer)
 - 3: **for each** epoch **do**
 - 4: **for each** mini-batch $(\mathbf{x}, \mathbf{v})$ **do**
 - 5: $\mathbf{h} \leftarrow M_{\text{frozen}}(\mathbf{x})$
 - 6: $\hat{\mathbf{v}} \leftarrow g_\phi(\mathbf{h})$
 - 7: $\ell \leftarrow \mathcal{L}(\mathbf{v}, \hat{\mathbf{v}})$
 - 8: Update ϕ via back-propagation and AdamW
 - 9: **end for**
 - 10: Evaluate $M \circ g_\phi$ on validation data
 - 11: **end for**
 - 12: **Output:** Fine-tuned regressor parameters ϕ^*
-

C. Fine-Tuning YaTC

We fine-tune the pre-trained YaTC model [7], [26], originally developed for traffic classification tasks, by adapting it on the regression task of predicting traffic flow volume from image-based representations of the traffic flows. The YaTC model is based on a Vision Transformer architecture consisting of 4 Transformer layers with 16 attention heads and an embedding dimension of 192. Each input is a grayscale image of size 40×40 , where rows and columns are structured to encode hierarchical flow features, including byte-level, packet-level, and flow-level information.

To adapt YaTC for regression, the original classification head is replaced with the regressor head (see Section IV-A). Only the regressor and patch embedding layers are updated during training, while all Transformer weights remain frozen. Pre-processing involves converting traffic flows into grayscale

images (2D matrix) and preparing corresponding float labels. The high-level algorithm is depicted in Algorithm 1.

D. Training a Lightweight Transformer

We design and train a lightweight Transformer model to perform flow volume regression. In contrast to NetFound and YaTC, this model does not rely on pre-training and is optimized end-to-end on our downstream task. Each input is a flow represented as a fixed-length sequence of two packets, where each packet is described using four features: IAT, packet size, packet direction, and protocol type. These features are extracted from raw PCAP files using a custom pre-processing pipeline. Sequences are padded or truncated to a fixed length 2, resulting in an input tensor of shape 2×4 for each flow.

A linear embedding layer first maps the 4-dimensional input vectors into a higher-dimensional hidden space. Positional encodings are then added to preserve the temporal order of packets, enabling the model to capture sequential dependencies. The resulting sequence is passed through a single Transformer encoder block, followed by mean pooling to derive a global representation of the flow. This representation is passed to the regression head (see Section IV-A), as shown in Algorithm 2.

Algorithm 2 Generic Supervised Training Loop for Scratch-Trained Models (MLP and Lightweight Transformer)

- 1: **Input:** Feature batches $\mathcal{X}$, volume labels $\mathcal{V}$, model $f_\theta \in \{\text{MLP, LW-Tr}\}$, loss $\mathcal{L}$
 - 2: **for each** epoch **do**
 - 3: **for each** mini-batch $(\mathbf{x}, \mathbf{v})$ **do**
 - 4: **if** f_θ is **LW-Tr** **then**
 - 5: $\mathbf{e} \leftarrow \text{Embed}(\mathbf{x})$
 - 6: $\tilde{\mathbf{e}} \leftarrow \text{AddPosEnc}(\mathbf{e})$
 - 7: $\mathbf{h} \leftarrow \text{TrEncoder}(\tilde{\mathbf{e}})$
 - 8: $\mathbf{z} \leftarrow \text{MeanPool}(\mathbf{h})$
 - 9: $\hat{\mathbf{v}} \leftarrow \text{Regressor}(\mathbf{z})$
 - 10: **else**
 - 11: $\hat{\mathbf{v}} \leftarrow f_\theta(\mathbf{x})$
 - 12: **end if**
 - 13: $\ell \leftarrow \mathcal{L}(\mathbf{v}, \hat{\mathbf{v}})$
 - 14: Update θ via back-propagation and AdamW
 - 15: **end for**
 - 16: Evaluate f_θ on validation data
 - 17: **end for**
 - 18: **Output:** Trained parameters θ^*
-

E. Training the MLP Regressor

We implement a simple MLP model (see Section IV-A) for flow volume regression. No pre-training or attention mechanism is used. Each flow is represented using a fixed-length sequence of two packets. For each packet, four features are extracted: IATs, packet size, protocol type, and direction. These values are standardized and flattened into a single vector of length 8 (i.e., 2 packets $\times$ 4 features). This vector serves as the input to the MLP model, as shown in Algorithm 2.

V. EXPERIMENTS

A. Datasets

We use two public network traffic datasets (*MAWI* and *CIC-IDS-2017*) to fine-tune NetFound and YaTC, and to train our proposed lightweight Transformer-based model and an MLP regressor. Both datasets provide raw traffic in the form of packet capture (PCAP) files. We split the raw PCAP files into bi-directional flows based on a 5-tuple (source IP, destination IP, source port, destination port, protocol). Each flow is terminated using an idle timeout of 5 seconds or an active timeout of 120 seconds, following the approach used in CICFlowMeter [27]. For each flow, we compute the total number of bytes transferred (flow volume) as the regression target, and extract early packet-level features. We randomly split the resulting flows into training and evaluation sets. These two datasets deliberately span complementary extremes: CIC-IDS-2017 offers a controlled, lab-crafted baseline, whereas MAWI represents high-volume, unconstrained backbone traffic. The differing results therefore highlight how model capacity and tokenisation choices interact with traffic heterogeneity. A broader multi-corpus evaluation—including enterprise, mobile and IoT traces—remains future work beyond the scope of this first study.

CIC-IDS-2017 [11], [12]: This dataset includes both benign and malicious traffic scenarios commonly used in intrusion detection research. In this study, we focus on the benign traffic collected during the “Monday” session, which provides a clean baseline of normal network behavior. For our regression task, we use only these benign flows.

MAWI [13], [14]: The MAWI Working Group Traffic Archive contains real-world backbone traffic captured from a Japanese academic network. We use PCAP trace files from April 2024.

Similar to CIC-IDS-2017, we construct flows using 5-tuple keys and extract packet-level features to model flow volume. Different pre-processing steps were applied depending on the model being trained:

- *NetFound*: We leveraged the official NetFound data pre-processing scripts to tokenize traffic flows into bursts and extract packet-based metadata. Each burst was converted into a sequence of feature tokens using the protocol-aware tokenizer.
- *YaTC*: Following the YaTC pipeline, each PCAP flow was first converted into a grayscale image using MFR method, which captures byte-level, packet-level, and flow-level features in a 2D matrix suitable for Vision Transformer input.
- *Lightweight Transformer*: For our model, we extracted four numerical features from the first two packets of each flow: IATs, packet size, protocol type, and direction. These values were zero-padded if fewer than two packets were available. These features were selected to capture both *temporal* characteristics (e.g., IATs) and *spatial* characteristics (e.g., packet sizes) of the flow.

- *MLP*: for the MLP model, we extracted the features of the first two packet (IATs, size, protocol, and direction) as a single vector.

B. Experimental Setup

All models are trained on the same machine equipped with an NVIDIA A2 GPU (16 GB), CUDA version 11.7, and NVIDIA GPU driver version 515.105.01. We train and evaluate four different architectures for flow volume regression task:

a) *NetFound*: We fine-tune the pre-trained netFound-640M-base model using the Adam optimizer and Mean Squared Error (MSE) loss function. The regression head is trained for 10 epochs with a learning rate of 0.0001 and a batch size of 40. The Transformer base model remains frozen during training. Inputs are burst-level features including tokenized packet content, direction, byte count, IATs, packet count, and protocol type.

b) *YaTC*: We fine-tune the pre-trained YaTC model for 50 epochs using the Adam optimizer and Mean Absolute Error (MAE) loss function, with a base learning rate of 2×10^{-3} and a batch size of 64. Each input is a 40×40 grayscale image representing one flow as a MFR matrix.

c) *Lightweight Transformer*: Our lightweight Transformer model is implemented using PyTorch Lightning library and optimized with the Adam optimizer and MSE loss function. It is trained for 50 epochs with a batch size of 64 and an initial learning rate of 10^{-3} . Each input is a 2×4 matrix capturing the IATs, packet size, direction, and protocol type of the first two packets in the flow. Implementation details are summarized in Table I.

TABLE I: Lightweight Transformer Training Configuration

Hyperparameter	Value
Learning rate	1×10^{-3}
Feature embedding size	32
Number of Transformer encoders	1
Number of attention heads	8
Feedforward layer size in encoder	512
Activation function in encoder	ReLU
Input sequence length	2 packets
Optimizer	Adam
Loss function	MSE
Batch size	64

d) *MLP*: The MLP model is trained using the Adam optimizer and MSE loss for 50 epochs with a batch size of 64 and a learning rate of 1×10^{-4} .

Table II summarizes the key architectural characteristics, input data, and parameter counts (trainable and non-trainable) for the four evaluated models.

C. Evaluation Metrics

To assess regression performance of the trained models, we employ four metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R^2). Additionally, we

TABLE II: Comparison of the Used Models

Model	Architecture	Input Features	Parameter Count
NetFound [6]	Hierarchical Transformer with 24 encoder layers, 1024 hidden size, 16 attention heads; MLP regressor head.	Tokenized packets (Up to 72 packets) (header + first 12 bytes of payload), direction, byte count, IATs, packet count, and protocol type	Total: 648M Trainable: 5.25M Frozen: 642.77M
YaTC [7]	Vision Transformer with 4 Transformer layers, 16 attention heads, 192 embedding dimension; MLP regressor head.	40×40 grayscale images per flow: 5 packets , each with 2 rows (80 bytes) for headers and 6 rows (240 bytes) for payloads.	Total: 1.93M Trainable: 75.3K Frozen: 1.86M
Lightweight Transformer	Transformer with 1 encoder layer, 8 attention heads, 32-dim embedding, feedforward size 512; MLP regressor head.	2×4 feature matrix (IAT, packet size, direction, protocol type) from first 2 packets .	Total: 348K Trainable: 348K Frozen: 0
MLP	Simple feedforward network with 3 linear layers, LeakyReLU activations and dropout.	Vector of 8 features (2 packets $\times$ [IAT, size, protocol, direction]).	Total: 35K Trainable: 35K Frozen: 0

use scatter plots as a visual tool to compare the prediction accuracy between the four trained models.

- *Mean Squared Error:*

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MSE measures the average squared difference between actual values y_i and predicted values $\hat{y}_i$ for n test samples. MSE is sensitive to outliers. Lower MSE indicates better performance.

- *Mean Absolute Error:*

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE calculates the average absolute difference between actual values y_i and predicted values $\hat{y}_i$. It shows how much the predictions deviate from the actual values. It is less sensitive to outliers compared to MSE. Lower MAE indicates more accurate predictions.

- *Mean Absolute Percentage Error:*

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

MAPE expresses prediction accuracy as a percentage, by showing what percentage is the prediction off from the true value, making it easier to interpret across datasets. Lower MAPE means higher accuracy.

- *Coefficient of Determination R^2 :*

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

R^2 indicates how much of the variation in the data is captured by the model. An R^2 of 1 indicates perfect predictions, while 0 means the model performs no better than predicting the mean.

- *Scatter Plots:* Scatter plots visualize the relationship between predicted and actual flow volumes. Each point on the plot represents a pair of actual and predicted values for a test sample, while perfect predictions lie along the diagonal line (the ideal-fit). The closeness of points to

this ideal-fit line qualitatively reflects prediction accuracy, highlighting the distribution of residuals (the difference between the actual value and the predicted value).

VI. RESULTS AND DISCUSSION

This section presents the performance of all four models across two datasets: MAWI and CIC-IDS-2017.

A. Numerical Results

Tables III and IV summarize the regression performance using four metrics: R^2 , MSE, MAE, and MAPE.

MAWI: The MAWI dataset captures real-world traffic from a backbone network, which introduces high variability in flow sizes and traffic patterns. In this challenging setting, the pre-trained NetFound model achieves the best performance with an R^2 of 0.812 and the lowest MSE, benefiting from its hierarchical attention, protocol-aware design and its rich input structure (including up to 72 tokenized packets). YaTC ranks second with $R^2 = 0.760$, showing the effectiveness of image-based representations. Lightweight and MLP models underperform here due to their limited ability to model flow patterns using minimal early-packet features.

CIC-IDS-2017: This dataset is less diverse, as it is generated in a controlled testbed environment. Here, the lightweight Transformer outperforms all others with an R^2 of 0.963, showing its suitability for early-packet-based predictions on this kind of traffic. YaTC and NetFound also perform well with R^2 values of 0.951 and 0.934, respectively. Even the MLP model achieves a strong $R^2 = 0.840$, highlighting that simple models can perform well in low-variance dataset.

B. Visual Results

Figures 2, 3, 5, and 4 illustrate the scatter plots of predicted $\hat{y}$ values vs. actual flow volumes y for each model. The red dashed line indicates the ideal $\hat{y} = y$ fit.

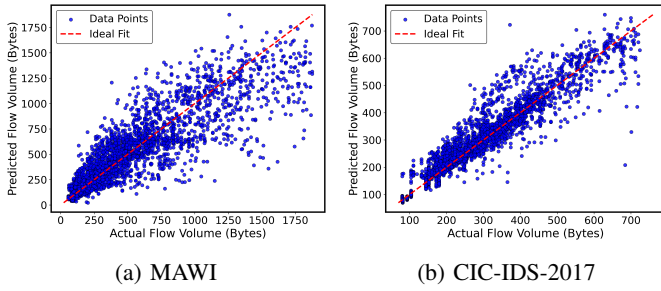
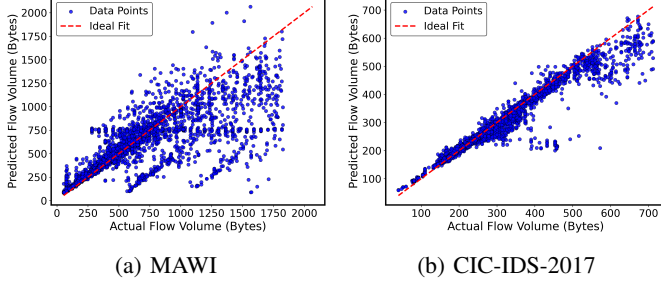
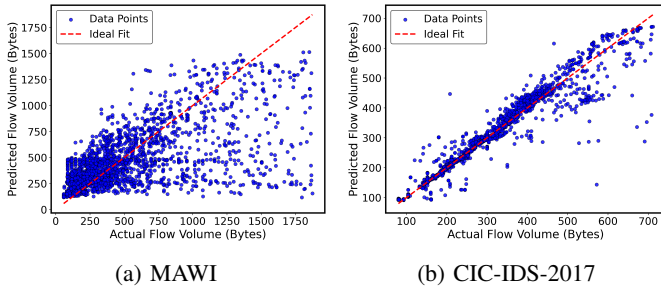
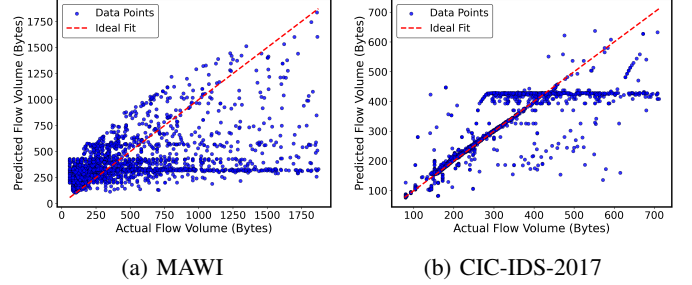
NetFound: On MAWI (Fig. 2a), the predictions generally follow the ideal trend but exhibit noticeable dispersion, particularly for higher flow volumes. This suggests that while NetFound benefits from pre-training and hierarchical attention, it still faces challenges in modeling the variability of real-world traffic. On CIC-IDS-2017 (Fig. 2b), the model performs

TABLE III: Performance Comparison on MAWI Dataset

Metric	NetFound [25]	YaTC [7]	Lightweight Transformer	MLP
$R^2 \uparrow$	0.812	0.760	0.50	0.281
MSE (Byte ²) $\downarrow$	23503.01	31934.47	6633092.00	88188.22
MAE (Byte) $\downarrow$	85.37	76.56	1549.82	200.93
MAPE (%) $\downarrow$	0.238	20.80	10.962	0.834

TABLE IV: Performance Comparison on CIC-IDS-2017 Dataset

Metric	NetFound [25]	YaTC [7]	Lightweight Transformer	MLP
$R^2 \uparrow$	0.934	0.951	0.963	0.840
MSE (Byte ²) $\downarrow$	1153.423	865.209	629.238	2723.600
MAE (Byte) $\downarrow$	18.18	14.36	9.89	21.63
MAPE (%) $\downarrow$	0.082	7.210	0.043	0.072

Fig. 2: Comparison of prediction accuracy of the fine-tuned **NetFound** model on (a) MAWI and (b) CIC-IDS-2017.Fig. 3: Comparison of prediction accuracy of the fine-tuned **YaTc** model on (a) MAWI and (b) CIC-IDS-2017.Fig. 4: Comparison of prediction accuracy of the trained **Lightweight Transformer** model on (a) MAWI and (b) CIC-IDS-2017.Fig. 5: Comparison of prediction accuracy of the trained **MLP** model on (a) MAWI and (b) CIC-IDS-2017.

more consistently, with predictions more tightly clustered along the ideal fit line, though it is slightly outperformed by the lightweight model and the Yatec model.

YaTC: The fine-tuned YaTC model exhibits reasonable alignment on MAWI dataset (Fig. 3a), although the spread around the ideal line is wider than NetFound, possibly due to loss of temporal information in image-based encoding, as YaTC does not encode packet timing such as IATs, or temporal order beyond a 5-packet sequence. On CIC-IDS-2017 (Fig. 3b), predictions are more tightly clustered, showing strong generalization.

Lightweight Transformer: On MAWI (Fig. 4a), predictions are more dispersed, especially for larger flows, indicating challenges in modeling highly variable real-world traffic using only early packet-level features. However, on CIC-IDS-2017 (Fig. 4b), the alignment is excellent. Predictions closely follow the ideal line.

MLP: (Fig. 5a) shows that the MLP struggles to generalize on MAWI dataset. Predictions are scattered with many predictions clustered around fixed values regardless of input. A similar pattern is observed on the CIC-IDS-2017 dataset (Fig. 5b), where despite better alignment for smaller flows, the model still shows prediction plateaus. This indicates that the MLP fails to capture flow volume patterns even in more structured datasets.

VII. LIMITATIONS AND FUTURE WORK

While this work offers the first empirical evidence that Transformer foundations can tackle flow-volume regression,

several limitations remain. We evaluated only two datasets: CIC-IDS-2017 and MAWI, which do not cover enterprise, mobile, or IoT environments. Each model used its native tokenisation pipeline, which reflects realistic usage but prevents a fully controlled comparison; future ablations will standardise feature sets to isolate architectural effects. Our baselines focused on Transformers and an MLP; future work will incorporate CNNs, and temporal-convolutional networks.

VIII. CONCLUSION

This paper investigated the use of Transformer models for predicting network flow volume, by fine-tuning NetFound and YaTC, introducing a lightweight Transformer, and evaluating their performance against a simple MLP model. The results demonstrated a clear advantage of Transformers, which consistently outperform MLPs, particularly on real-world traffic like MAWI, by effectively uncovering deeply patterns in the traffic data. These findings shows the remarkable capacity of Transformers to learn rich representations from early packet-level features, making them highly suitable for regression tasks in network traffic analysis.

ACKNOWLEDGMENT

This work was supported by HORIZON-MSCA-2022-SE-01 under the project TRACE-V2X (grant agreement ID 101131204).

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [2] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, C. Zhang, X. Wang, and J. Liu, "Large language model (llm) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2024.
- [3] M. A. Habib, P. E. I. Rivera, Y. Ozcan, M. Elsayed, M. Bavand, R. Gaigalas, and M. Erol-Kantarci, "Transformer-based wireless traffic prediction and network optimization in o-ran," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 1–6.
- [4] M. A. Ferrag, M. Ndhlovu, N. Tihanyi, L. C. Cordeiro, M. Debbah, T. Lestable, and N. S. Thandi, "Revolutionizing cyber threat detection with large language models: A privacy-preserving bert-based lightweight model for iot/iiot devices," *IEEE Access*, vol. 12, pp. 23 733–23 750, 2024.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [6] S. Guthula, R. Beltiukov, N. Battula, W. Guo, and A. Gupta, "netfound: Foundation model for network security," *arXiv preprint arXiv:2310.17025*, 2023.
- [7] R. Zhao, M. Zhan, X. Deng, F. Li, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 2012–2025, 2024.
- [8] H. Wen, P. Sharma, V. Yegneswaran, P. Porras, A. Gehani, and Z. Lin, "6g-xsec: Explainable edge security for emerging openran architectures," in *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, ser. HotNets '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 77–85. [Online]. Available: <https://doi.org/10.1145/3696348.3696881>
- [9] A. Garza, C. Challu, and M. Mergenthaler-Canseco, "Timegpt-1," *arXiv preprint arXiv:2310.03589*, 2023.
- [10] M. Abbasi, A. Shahraki, and A. Taherkordi, "Deep learning for network traffic monitoring and analysis (ntma): A survey," *Computer Communications*, vol. 170, pp. 19–41, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0140366421000426>
- [11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "CICIDS2017: A realistic dataset for Intrusion Detection Systems," <https://www.unb.ca/cic/datasets/ids-2017.html>, 2018, canadian Institute for Cybersecurity.
- [12] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani *et al.*, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, no. 2018, pp. 108–116, 2018.
- [13] M. W. Group, "The mawi (measurement and analysis on the wide internet) working group) working group traffic archive," <https://mawi.nezu.wide.ad.jp/mawi/ditl/ditl2024/>, 2024.
- [14] I. Tsareva, T. V. Doan, and V. Bajpai, "A decade long view of internet traffic composition in japan," in *2023 IFIP Networking Conference (IFIP Networking)*, 2023, pp. 1–9.
- [15] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.
- [16] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, ser. WWW '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 633–642. [Online]. Available: <https://doi.org/10.1145/3485447.3512217>
- [17] Q. Wang, C. Qian, X. Li, Z. Yao, G. Zhou, and H. Shao, "Lens: A foundation model for network traffic," *arXiv preprint arXiv:2402.03646*, 2024.
- [18] Z. Zhao, H. Chen, J. Zhang, X. Zhao, T. Liu, W. Lu, X. Chen, H. Deng, Q. Ju, and X. Du, "Uer: An open-source toolkit for pre-training models," *EMNLP-IJCNLP 2019*, p. 241, 2019.
- [19] J. López Espejel, E. H. Ettifouri, M. S. Yahaya Alassan, E. M. Chouham, and W. Dahhane, "Gpt-3.5, gpt-4, or bard? evaluating llms reasoning ability in zero-shot setting and performance boosting through prompts," *Natural Language Processing Journal*, vol. 5, p. 100032, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2949719123000298>
- [20] M. Movahedi and J. Choi, "The crossroads of llm and traffic control: A study on large language models in adaptive traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 2, pp. 1701–1716, 2025.
- [21] Hugging Face, "Hugging face: The ai community building the future," 2025, accessed: 2025-04-09. [Online]. Available: <https://huggingface.co>
- [22] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [23] T. Song, P. Garza, M. Meo, and M. M. Munafò, "Dex: Deep learning-based throughput prediction for real-time communications with emphasis on traffic extremes," *Computer Networks*, vol. 249, p. 110507, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128624003396>
- [24] X. Wang, Z. Wang, K. Yang, Z. Song, C. Bian, J. Feng, and C. Deng, "A survey on deep learning for cellular traffic prediction," *Intelligent Computing*, vol. 3, p. 0054, 2024. [Online]. Available: <https://spj.science.org/doi/abs/10.34133/icomputing.0054>
- [25] S. UCSB, "netfound-640m-base," <https://huggingface.co/snlucsb/netFound-640M-base>, 2025, accessed: 2025-05-05.
- [26] NSSL-SJTU, "Yatc: Yet another traffic classifier," <https://github.com/NSSL-SJTU/YaTC>, 2023, accessed: 2025-05-05.
- [27] A. H. Lashkari, "Cicflowmeter: Network traffic flow generator," <https://github.com/ahlashkari/CICFlowMeter>, 2017, accessed: 2025-05-11.